Computers and Electrical Engineering 123 (2025) 110235

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

A lightweight model for distracted driver detection based on neural architecture search and coordinate attention

Haibin Sun, Mengting Zhang *

College of Computer Science and Engineering, Shandong University of Science and Technology, 579 Qianwangang Road, QingDao, 266590, Shandong Province, PR China

ARTICLE INFO

Dataset link: https://kaggle.com/competitions/state-farm-distracted-driver-detection, https://heshameraqi.github.io/distraction_detection

Keywords:
Distracted driving
Coordinate attention
Neural architecture search
CNN

ABSTRACT

Research indicates that driver distraction is a leading cause of most accidents. The distracted driver recognition task can be modeled using deep learning, and the lightweight deep learning model has been extensively studied for this task. In this study, we designed a novel lightweight network by combining a neural architecture search and the coordinate attention mechanism (CA) based on the characteristics of the distracted driver recognition task. The designed neural architecture searching space was searched to find the most suitable network for recognizing distracted driver behaviors. This model, combined with CA, forms a new lightweight framework specifically designed for distracted driver recognition, called CondConv coordinate attention network (CCNET). Experimental evaluations were conducted on two public datasets: AUC distracted driver and State Farm distracted driver detection. The accuracy of CCNET for AUC was 95.52%, whereas it ranged from 99.8% to 99.9% for the State Farm. To simulate real driving scenarios, the model was deployed on an embedded device (Nvidia Jetson Nano B01), achieving a real processing speed of 54 frames per second and enabling real-time processing. These experimental results indicate that the performance of CCNET is superior to that of most previous lightweight models. Notably, the CCNET model has only 1.7 million parameters and an efficient inference speed, making it smaller and faster than most previous lightweight models.

1. Introduction

Traffic accidents are anticipated to increase as transportation systems continue to evolve. A report from the Indian Ministry of Road Transport and Highways revealed that in 2022, India witnessed 461,312 road accidents, leading to 168,491 fatalities and 443,366 injuries [1]. These figures underscore the significant social and economic toll that road traffic accidents impose on countries and societies.

Distracted driving accounts for approximately 16% of societal harm and is a significant contributing factor to traffic accidents. The National Highway Traffic Safety Administration (NHTSA) estimated that approximately 15% of traffic accidents are caused by distracted driving [2]. Furthermore, it is reported that 'distraction-affected incidents' comprise 10% of accidents and 18% of injury-causing accidents. According to the NHTSA report, distracted driving encompasses activities such as texting, calling, adjusting audio systems, conversing with passengers, eating, and drinking [2]. Therefore, the detection of distracted driving behavior is of paramount importance and can significantly contribute to reducing the incidence of road accidents. For example, an Uber autonomous vehicle was involved in a fatal accident because of its inability to identify a pedestrian [3].

* Corresponding author.
E-mail addresses: skd992989@sdust.edu.cn (H. Sun), zmt@sdust.edu.cn (M. Zhang).

https://doi.org/10.1016/j.compeleceng.2025.110235
Received 26 April 2024; Received in revised form 13 February 2025; Accepted 27 February 2025
Available online 11 March 2025
0045-7906/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Thus, distracted driving requires considerable attention, and prompt detection of such behavior can trigger preventative measures. This underscores the importance of advanced driver assistance systems and intelligent transportation systems in identifying distracted driving behaviors. These intelligent systems depend predominantly on computer vision and image detection technologies.

With the continuous advancement of deep learning technology, its widespread application has become particularly notable in recent times, especially in the fields of object detection and image classification. Deep learning models demonstrate superior performance compared with traditional algorithms. This trend has prompted the development of distracted driver detection models using convolutional neural networks (CNNs). However, most models employ complex network architectures such as AlexNet [4], VGG [5], GoogleNet [6], ResNet [7], and InceptionResNetV2 [8], which typically have a large number of parameters. The deployment of these large-scale models in practical applications is challenging because of the limited memory and power capacities of embedded devices. This study proposes a lightweight model that achieves high accuracy in distracted driver detection with fewer parameters.

The contributions of this study are as follows:

1. This study analyzes and designs a neural architecture search (NAS) space [9] tailored to the characteristics of the driver behavior recognition task. Using this search space, a novel model is generated through a NAS algorithm, achieving high accuracy with minimal parameters.
2. Second, to improve the efficiency of the model in extracting and utilizing essential features, this study introduces the recently proposed coordinate attention [10] (CA) mechanism, which improves the accuracy while avoiding significant computational overhead.
3. The experimental results show that our model outperforms most existing state-of-the-art models, achieving high accuracy with only 1.7 million parameters. Additionally, it demonstrates excellent real-time processing capability, maintaining a speed of 54 frames per second (FPS) on an embedded device.

The remainder of this paper is organized as follows. The next section introduces related research. Section 3 introduces the key technologies employed in this study. In Section 4, we present the proposed model. Section 5 introduces the datasets used, and discusses the experimental results. Section 6 summarizes the experiments and outlines future work.

2. Related work

This section reviews relevant studies on distracted driver detection and the development for distracted driver detection. Smartphone use while driving has been identified as a leading cause of distracted driving [11]. In the 1990s, researchers began developing datasets to detect phone usage behind the wheel. Zhang et al. [12] proposed capturing images of driver behavior by mounting cameras on dashboards. The authors used images of hands, facial expressions, and the mouth to determine and detect phone usage.

Das et al. [13] introduced a comprehensive, publicly available, and natural video-based hand dataset. They proposed the aggregate channel feature detector method, achieving an accuracy of 70.09%. Since this method relies on hand datasets for recognition, it overlooks key information such as the head and steering wheel, which results in lower recognition accuracy. A dataset was created by Seshadri et al. [14] to detect driver phone use, reaching 93.9% classification accuracy using techniques such as the histogram of oriented gradients (HOG) and the supervised descent method (SDM). However, the applicability of this method in complex environments still presents challenges. While HOG features perform excellently in simple texture tasks, their feature extraction ability is limited under conditions such as lighting variations, occlusions, or different viewing angles, which can lead to a decline in classification performance. A faster RCNN using this dataset was proposed by Le et al. [15], which achieved an accuracy of 94.2%. Although the model demonstrates an improvement in detection accuracy, its computational complexity remains high, making it difficult to meet the deployment requirements of embedded devices or real-time systems.

A more extensive driving dataset was created by Zhao et al. [16], comprising safe driving and using a mobile phone to call, eat, and operate the shift lever. They used various filtering techniques, skin region segmentation, and contourlet transform for feature extraction, achieving 90.5% accuracy on the Southeast University (SEU) dataset with a random forest classifier. Yan et al. [17] introduced a CNN-based method that achieved a classification accuracy of 99.78% on the SEU dataset.

These methods focus on restricted datasets, encompassing only a few distracted behaviors, and most of these datasets are not accessible to the public. Traditional distracted driver detection algorithms employ hand-crafted features, such as hand, facial, and head posture features. The State Farm distracted driver detection dataset [18], introduced in 2016, was the first to encompass a diverse range of distracted behaviors and was made publicly accessible. Many solutions have been proposed for this dataset, with CNNs demonstrating superior effectiveness compared with traditional algorithms [19].

In 2017, Abouelnaga et al. [20] introduced the American University in Cairo dataset, which is similar to the State Farm dataset. They made it publicly available and proposed a weighted ensemble of five CNNs. However, the large network size made embedded deployment challenging. Baheti et al. [21] proposed a modified VGG16 method that achieved an accuracy of 95.54%. Zhao et al. [22] proposed an enhanced AlexNet and compared it with ResNet. The aforementioned studies indicate that earlier research primarily focused on improving accuracy, but these large networks are impractical for in-vehicle computing.

Recently, as deep learning has advanced in the fields of image segmentation, recognition, and object detection, various models, such as AlexNet, VGG, GoogLeNet, ResNet, and InceptionResNetV2, have demonstrated excellent performance in detecting abnormal driver behavior. However, these models primarily aim to improve the accuracy while overlooking the parameter size. Therefore,

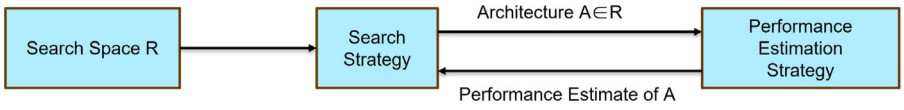


Fig. 1. Visual representation of neural architecture search methods.

researchers are now focusing on the development of lightweight high-precision models. In 2017, Howard et al. [23] introduced MobileNet, a lightweight model designed for mobile and embedded applications. By reducing the number of parameters through depthwise separable convolutions, MobileNet outperformed other popular models in the ImageNet classification task, achieving higher accuracy with fewer parameters. In 2018, Sandler et al. proposed MobileNetV2 [24], which has an inverted residual structure and fewer parameters. SqueezeNet [25] relies on pruning and small convolutional kernels to reduce the network scale. Although this approach effectively lowers the computational burden, it results in reduced accuracy. Nevertheless, these lightweight models were not explicitly designed for distracted driver recognition, leaving room for improvement in accuracy and parameter size. Some models, such as MobileVGG [26], have been manually designed for distracted driver recognition. In 2021,DHCNN [27] further reduced the number of parameters using a structure with four convolutional blocks and achieved high performance with a small number of filters. However, it requires image preprocessing using HOG. In 2023, Chen et al. [28]proposed a lightweight integrated model that combined elements from the Inception, Xception, DenseNet, and MobileNet architectures. This model achieved an accuracy of 99% on the State Farm dataset. However, the scale and complexity of the model rendered it unsuitable for deployment on embedded devices. In 2023,Mittal et al. [29] proposed the Convolutional Attention Capsule network (CAT-CapsNet), which combines a capsule network with an attention module and convolution filters. In 2024, Gao et al. [30] proposed a model that combined CA with MobileNetV2, forming the MobileNetV2-CA model. However, these models were designed manually based on experience, and the maximum potential of the network structure could not be reached. Thus, we propose using NAS to evaluate and select the most promising network models by concatenating different structures and modules.

3. Preliminary work

In this section, we present the key technologies on which our architecture builds: (i) NAS, (ii) conditionally parameterized convolutions (CondConv) [31], and (iii) CA mechanisms.

3.1. Neural architecture search

NAS constructs a search space based on experience [9,32]. The core idea of NAS is to define a search space and use a search algorithm to generate a series of network architectures.

NAS includes three main steps: defining a searching space, executing a search strategy to sample networks within the search space, and evaluating the performance of the sampled networks. NAS approaches commonly define a search space that includes various operations such as convolution, pooling, and connected layers. Subsequently, possible network combinations are evaluated using a specific assessment strategy. The entire search process is controlled by reinforcement learning [33], evolutionary search [34], differentiable search [35], and other learning algorithms [36] that guide the search in directions that lead to better performance. The NAS approach helps to discover networks that excel in specific tasks. As shown in Fig. 1, in each iteration of the search process, ‘samples’ are generated from the search space, obtaining a neural network structure ‘A’ referred to as a ‘subnetwork’. A was trained on the training dataset and its performance was evaluated on the validation dataset. The network structure was gradually optimized until an optimal subnetwork was obtained. This paper adopts a differentiable search algorithm [35]. The search strategy involves designing a neural architecture search space, evaluating candidate networks on the AUC dataset [20].

3.2. CondConv

Yang et al. [31] proposed a method to parameterize convolutional kernels as a linear combination of n experts represented by $(\alpha_1 W_1 + \cdots + \alpha_n W_n) * x$. In this representation, α_1 to α_n are the weights obtained by calculating the routing function, n denotes the total number of experts (or convolutional kernels), and W_1 to W_n denote the parameters of each convolutional kernel within the CondConv structure. The variable x denotes the input features. As illustrated in Fig. 2, a CondConv with three experts is shown, where W_1, W_2, and W_3 represent the parameters of each convolutional kernel. This method offers improved computational efficiency compared to alternatives such as adding more convolutional layers or increasing the convolutional kernel size. This is because all experts are combined only once for each input, and this combined entity necessitates a single convolution operation, thereby optimizing the computational resources. Each convolutional kernel only needs to be computed once but is applied at many different positions in the input image. This means that by increasing n, we can increase the capacity of the network with only a small increase in inference cost.

Real-time detection of distracted drivers requires a high-capacity model capable of real-time processing. Consequently, reducing computational complexity during inference is of paramount importance. The enhanced accuracy of existing CNNs is primarily attributed to the enlargement of model sizes and use of larger datasets. However, further expansion of the model complicates the deployment process. Therefore, for tasks related to distracted driver detection, CondConv can serve as substitutes for the standard convolutions used in previous CNNs. In this study, this type of convolution is utilized as a key constituent convolution within the search space.

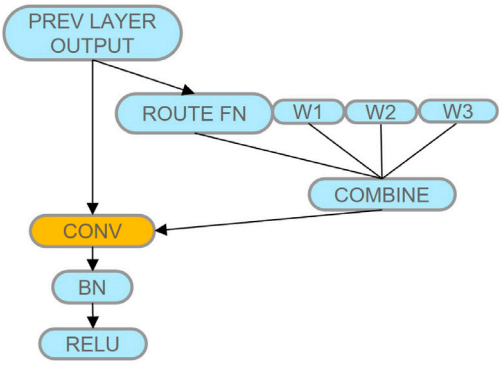


Fig. 2. CondConv layer architecture with $n = 3$ kernels.

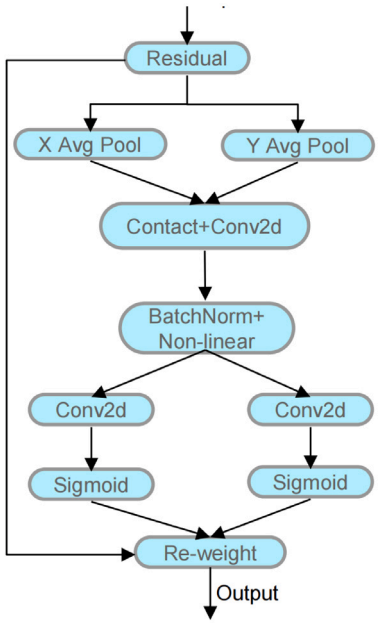


Fig. 3. 'X Avg Pool' and 'Y Avg Pool' represent 1D feature encoding in two directions (vertical and horizontal) respectively.

3.3. Coordinate attention

Given the relatively limited number of parameters in our model, it may face certain constraints in learning complex patterns and features, thus restricting its representational capacity. To mitigate this limitation, we introduce an attention mechanism that enhances the model's ability to focus on and extract salient features from regions of interest while effectively reducing interference from other, less relevant features. Additionally, due to the potential loss of critical information during the pooling process, the attention mechanism dynamically adjusts the weights of feature regions in accordance with the specific demands of the task, thereby strengthening the representation of key information. Consequently, we have opted to incorporate the attention mechanism after the pooling layer, thereby addressing the issue of information loss more effectively and enhancing the overall performance of the model.

Hou et al. [10] proposed the CA module in 2021 . Compared with traditional attention mechanisms such as Squeeze-and-excitation networks (SE) [37] and Convolutional block attention module (CBAM) [38], the CA module exhibits superior performance in deep learning applications. This is primarily because traditional channel-attention mechanisms employ global pooling to encode spatial information, which often results in loss of positional information. To address this issue, we decomposed global pooling into 1D feature encodings by using pooling kernels along the vertical and horizontal directions to encode each channel along the horizontal and vertical axes separately. This approach preserves the global spatial information and enhances the position sensitivity. The CA module is shown in Fig. 3.

In this study, the CA was incorporated following the pooling layer in each layer of the CNET. This is used to extract more valuable features and enhance the accuracy of the model for distracted driver detection.

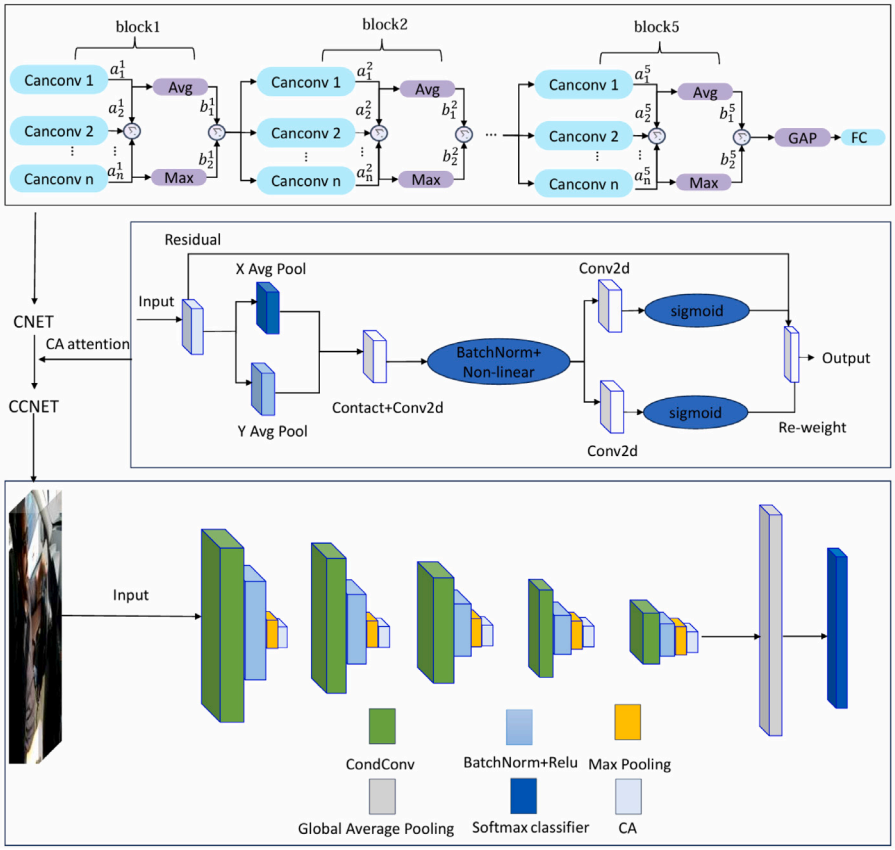


Fig. 4. Overview of the model, where 'CanConvn' is candidate CondConv. 'X Avg Pool' and 'Y Avg Pool' represent 1D feature encoding in two directions (vertical and horizontal) respectively.

4. Proposed method

In this section, the proposed model is described. Our primary emphasis is on the formulation of a lightweight model characterized by its superior accuracy and minimal parameter count. The architecture of the proposed model is shown in Fig. 4. The search space comprised candidate blocks for differently parameterized conditional convolutions (CanConv1 to CanConvn) and two pooling varieties (max pooling and average pooling). The output from each candidate block (CanConv and pooling) was weighted and aggregated to serve as the input for the subsequent candidate block. The learnable weights (a and b) represent the probabilities of selecting a candidate block.

Within each layer, the convolution and pooling blocks with the highest probabilities were selected. Along with the global average pooling layer and fully connected operations, these constitute the final CNET. To further enhance the model, a CA block was incorporated following the pooling layer, resulting in the formation of the CCNET architecture.

The network construction was divided into two main components: (i) a novel framework, CNET, derived through NAS, and (ii) the integration of CA behind the max-pooling layer of each level in the CNET network, resulting in CCNET.

4.1. CNET architecture

In the distracted driver recognition task, images exhibit higher similarity compared to those in other dataset. For instance, the ImageNet Dogs dataset, which involves classifying different dog breeds, contains images with various distinguishing features, such as fur color and body size. By contrast, in distracted driver recognition tasks, datasets are primarily collected by the same individual in a consistent setting (driving the same vehicle). Consequently, many visual features in distracted driver detection tasks do not provide valuable classification information. For instance, features such as fur color, body size, and ear position in the ImageNet Dogs dataset can serve as discriminative elements for classification. However, background information, such as driving context, skin color, and clothing, becomes insignificant in distracted driver recognition tasks. Consequently, these tasks do not require numerous object detectors. Instead, they concentrate on identifying certain discriminative features, such as head posture, hand posture, and position of the steering wheel. In convolutional neural networks, the network depth influences flexibility, with each channel functioning as an

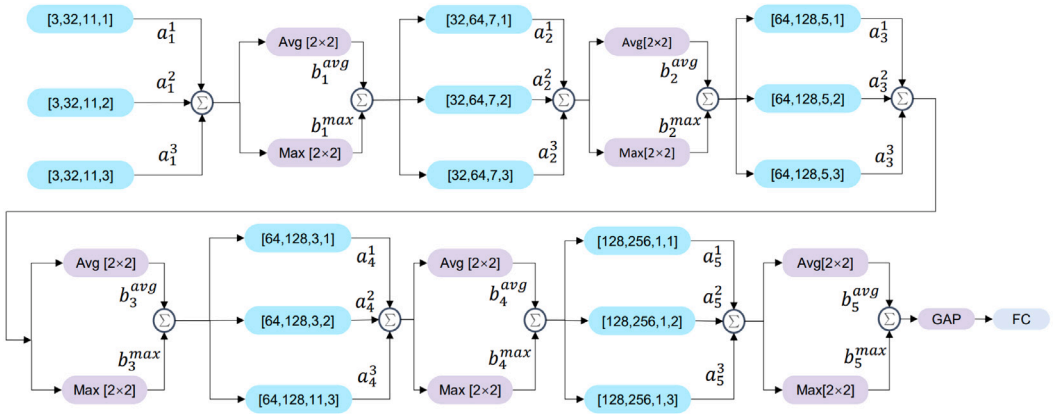


Fig. 5. Simplified visualization of the search process for the proposed CNET architecture.

object detector. Consequently, the distracted driver recognition task employs a comparatively shallow network structure comprising five convolutional layers, a global pooling layer, and fully connected layers.

Distracted driver detection tasks require a high-capacity model capable of real-time processing. Consequently, minimizing the computational complexity during inference becomes a critical task. Therefore, we used CondConv. Each of the five convolutional blocks comprised differently parameterized CondConvs. At the conclusion of each layer, pooling layers were employed to down-sample the feature maps, with an emphasis on utilizing both max pooling and average pooling. Therefore, we defined the search space as the candidate blocks for the differently parameterized CondConvs and two types of pooling within the five convolutional blocks. Owing to factors such as lighting and shadows, the AUC dataset is particularly demanding, and existing lightweight networks cannot achieve very high accuracy. Hence, our approach involves sequentially training individual candidate networks and assessing their performances on the AUC validation set. Subsequently, the evaluation results were used to update the architecture search process [39].

The defined search space is illustrated in Fig. 5, where each candidate block comprises convolutional kernels with different sizes and numbers of experts (convolutional kernels). For example, $[3, 32, 11, 1]$, $[3, 32, 11, 2]$, and $[3, 32, 11, 3]$ represent three candidate blocks in a convolutional layer with the difference being the number of experts. For example, $[3, 32, 11, 3]$ denotes a convolution with a kernel size of 11 and three experts. The pooling layers are illustrated by window size and pooling type. For example, Avg Pool $[2 \times 2]$ denotes the average pooling with a stride size of 2, and the padding size is set to $\frac{\theta-1}{2}$, where θ is the spatial size of the filter. The output of each candidate block was weighted and aggregated to form the input for the next candidate block. Weights (a) and (b), which are learnable, represent the probabilities of being selected for the candidate block. The AUC dataset was used to train the network, and the loss was computed. Finally, by selecting the convolutional kernel layer and pooling layer combination with the highest probability for each layer, the final network, called CNET, is formed. The entire network was divided into five layers. The value of b indicates the number of current layers, and num represents the number of experts (number of convolutional kernels), a_b^{num}, where $b = \{1, 2, 3, 4, 5\}$ and $num = \{1, 2, 3\}$ represent the weight of CondConv with num convolutional kernels. b_b^{max} and b_b^{avg} represent the weights of the max pooling and average pooling in the b-th layer, respectively. GAP represents the global average pooling, while FC denotes fully connected pooling. After the completion of the search, only the weights with the maximum values in each layer were retained to form a new network.

$$X_b^{\text{out}} = b_b^{\max}\left(\sum_{m=1}^{3} a_b^m\left(X_b^{\text{in}}\right)\right) + b_b^{avg}\left(\sum_{m=1}^{3} a_b^m\left(X_b^{\text{in}}\right)\right) \tag{1}$$

where x_b^{in} is the input feature map for the b-th layer and x_b^{out} is the output feature map for the current b-th layer. The weights $\{a_b^1, a_b^2, a_b^3\}$ represent the probability weights of the convolutional kernels in the b-th layer, and the numbers of experts are one, two, and three. $\{b_b^{avg}, b_b^{max}\}$ denote the probability weights of average pooling and max pooling in the b-th layer. $\{a_b^1, a_b^2, a_b^3\}$ and $\{b_b^{avg}, b_b^{max}\}$ represent the probabilities of selecting the corresponding blocks, and they are computed as follows:

$$a_b^{num} = \frac{\exp\left(a_b^{num}\right)}{\Sigma_{j=1}^{3}\exp\left(a_b^{j}\right)} \tag{2}$$

$$b_b^{pool} = \frac{\exp\left(\mathrm{b}_b^{pool}\right)}{\exp\left(\mathrm{b}_b^{\mathrm{avg}}\right) + \exp\left(\mathrm{b}_b^{\max}\right)}, \mathrm{pool} = \{\mathrm{avg}, \max\} \tag{3}$$

Table 1
The formation process of CCNET.

Algorithm 1 CCNET Algorithm

Input: AUC dataset

Step 1: Initialize the weights a_b^{num} and b_b^{pool}, where $b = \{1,2,3,4,5\}$, $num = \{1,2,3\}$, and $pool = \{avg, max\}$
for each epoch **do**
  for each layer b **do**
    Compute the output feature map X_b^{out} using Eq. (1)
    Update the weights a_b^{num} and b_b^{pool} using Eqs. (2) and (3)
  end for
  Compute the loss using the AUC dataset
  Update the network parameters using backpropagation
end for
Step 2: Select the convolutional kernel layers and pooling layers combination with the highest probability for each layer to form the network CNET.Add CA block after the pooling layer to form final network CCNET
return CCNET

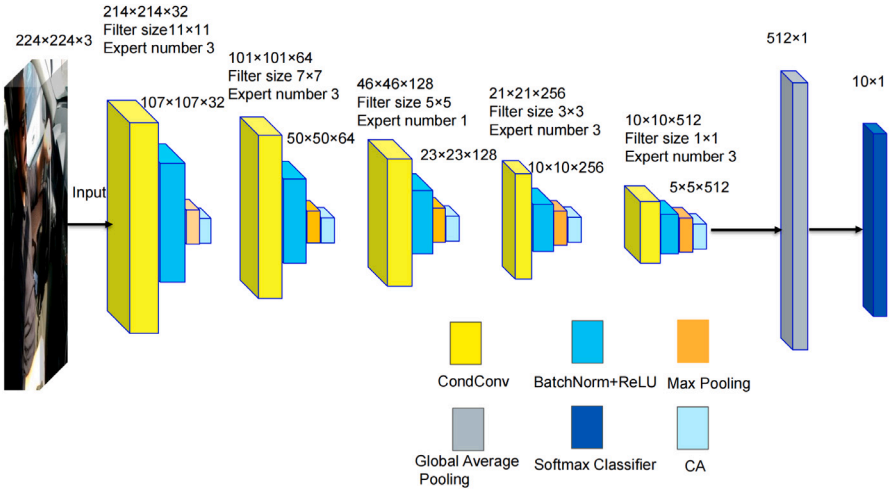


Fig. 6. Visualization of CCNET architecture where different colors represent various types of operations.

4.2. CCNET architecture

In contrast to channel attention, which only emphasizes the reweighting of various channel significances, the CA block incorporates the encoding of spatial information along the vertical and horizontal directions. Through this encoding process, the CA achieves more accurate localization of the object, ultimately enhancing the model's overall recognition capabilities. Incorporating CA is advantageous for enhancing the network's perception of discriminative features and improving the efficiency of extracting and utilizing important features.

Consequently, we added a CA mechanism after each pooling layer in CNET, resulting in the formation of our model, CCNET, as illustrated in Fig. 6. The algorithm of CCNET is listed in Table 1. The CCNET architecture comprises five convolutional layers. To ensure the network's performance and stability, batch normalization was employed after each convolutional layer, followed by activation via the ReLU function and subsampling through 2×2 pooling with a stride of two, thereby reducing the computation. The first layer uses a 11×11 filter with three experts, the second layer uses a 7×7 filter with three experts, the third layer uses a 5×5 filter with one expert, the fourth layer uses a 3×3 filter with three experts, and the fifth layer uses a 1×1 filter with three experts. CA is introduced after each pooling layer, which simultaneously considers channel relationships and positional information. This makes the model more sensitive to discriminative features. Finally, classification was performed using a global pooling layer, followed by Softmax.

5. Experimental study

5.1. Dataset description

The experiment employed the two most widely used public datasets on distracted driving currently available, each comprising one category representing normal driving and nine categories of distracted driving.



Fig. 7. Sample images of distracted drivers from the State Farm dataset.

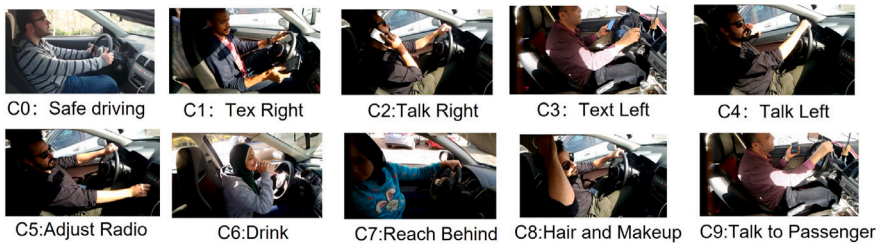


Fig. 8. Sample images of distracted drivers from the AUC dataset.

5.1.1. State Farm

The State Farm distracts the driver detection dataset [18], as shown in Fig. 7 was released by Kaggle and includes participants from diverse ethnic backgrounds across Asia and Africa. To collect the dataset, cameras were mounted on the dashboard to capture images of the drivers. To emulate genuine driving conditions, a truck towed the car, while the participants simulated ten distinct types of driving behaviors within the vehicle. Each behavior is represented by approximately 2000 images: (0) Safe driving, (1) Texting Right, (2) Talking Right, (3) Texting Left, (4) Talking Left, (5) Adjusting Radio, (6) Drinking, (7) Reaching Behind, (8) Hair and Makeup, and (9) Talking to Passenger. Each image has a resolution of 640 × 480 pixels.

The dataset comprised 22,424 labeled training images and 79,728 unlabeled test images. Consequently, experiments were conducted exclusively on the training dataset. The State Farm training dataset was distributed into training and testing sets at 90%–10%, 80%–20%, 75%–25%, and 70%–30%. In the second set of experiments, the State Farm dataset was randomly divided into a 60% training set, 20% validation set, and 20% test set for cross-validation experiments.

5.1.2. AUC

In 2017, Abouelnaga et al. [20] created a dataset similar to State Farm for distracted driver detection at the American University in Cairo (AUC), as shown in Fig. 8. The dataset comprised 31 participants from seven countries. Videos were recorded in four different cars, introducing variations in both drivers and driving conditions. Subsequently, the recorded videos were segmented into frames, each of size 1080 × 1920 pixels. The dataset was divided into a training set comprising 12,977 images, and a validation set containing 4331 images.

5.2. Results and discussion

This section is divided into two parts. The first part used NAS to discover the network structure, CNET, combined with CA to form the CCNET network. In this part, the learning rate was set to 0.002, and the number of epochs and batch size were set to 300 and 8, respectively. The algorithm was implemented in PyTorch. Implementation details are provided in Table 2. The search process took approximately 18 h. In the second part, experiments were conducted using CCNET on the State Farm and AUC datasets, and comparisons and discussions are presented with state-of-the-art approaches. The results indicate that CCNET exhibits low computational complexity and memory requirements while achieving good accuracy in driver posture classification. Stochastic gradient descent was used as the optimizer. All learning rates were set to 0.002 by employing a cosine-annealing learning rate strategy [40]. The numbers of epochs and batch sizes were 400 and 32, respectively. The algorithm was implemented in Pytorch on NVIDIA GeForce RTX 3060 with 16 GB of system RAM.

The original resolution of the raw images in the AUC distracted driver dataset was 1080 × 1920 × 3, whereas that in the raw image resolution of the State Farm dataset was 640 × 480. Before inputting them into the model, all images were resized to 256 × 256 × 3 using a bilinear interpolation. The resized photos were applied with a random crop of 224 × 224 region for training and the center crop of 224 × 224 region for testing. Random cropping helps reduce the model's excessive focus on certain local regions of the images. It also allows images to be cropped to arbitrary sizes, adding diversity to the dataset, and thereby enhancing the robustness and generalization capabilities of the model.

Table 2
System configuration.

Component	Details
Operating System	Windows 10
Programming Language	Python 3.10
Framework	Pytorch 2.1.0+cuda 11.8
CPU	AMD Ryzen 7 6800H, 3.20 GHz
GPU	NVIDIA GeForce RTX 3060 (6 GB)
Memory	16 GB

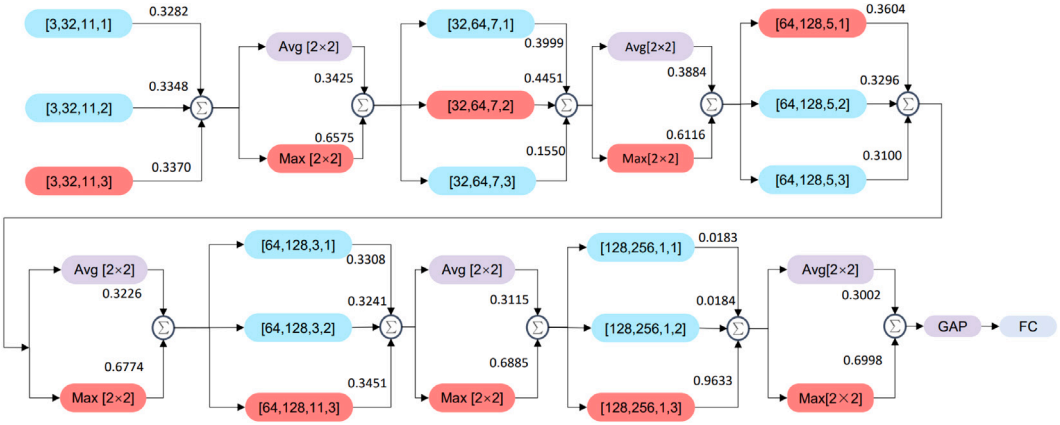


Fig. 9. Selection of probabilities for each candidate block.

5.2.1. *Neural architecture search for CNET*

We defined the search space as five convolutional blocks, each with a different design of CondConv. Each convolutional block in each layer was followed by two types of pooling layers: max pooling and average pooling. The network structure is illustrated in Fig. 9. Given the challenges of the AUC dataset, training and evaluation were performed using the AUC dataset. The experimental results are presented in Fig. 9, where the output numbers represent the probabilities of selecting each candidate block and the candidate block with the highest probability in each layer. The selected convolutional and pooling layers for each layer are indicated in red. The selected convolutional and pooling layers form a new network called CNET. CA is incorporated after the maximum pooling layer of CNET, resulting in the CCNET network, as shown in Fig. 6.

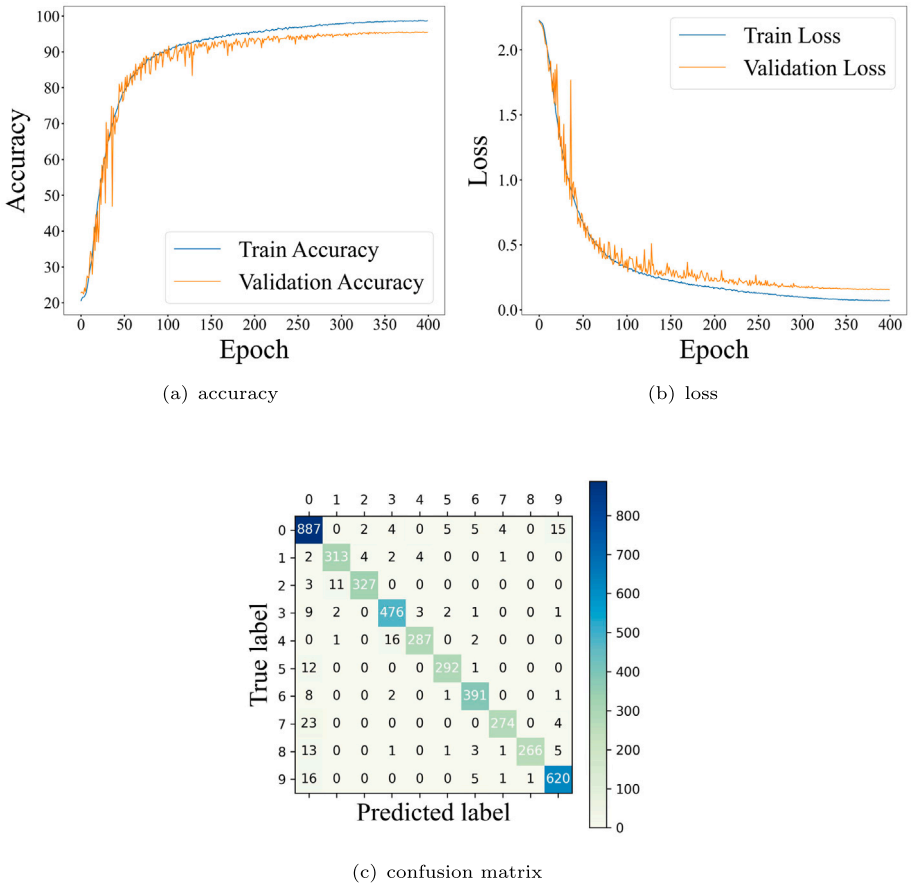
5.2.2. *AUC dataset results and discussion*

For a comparison with previous methods, when conducting experiments on the AUC dataset, we followed the partitioning method reported by Abouelnaga et al. [20], who introduced this dataset. According to the author's division, the training set comprises 12,977 images, and the validation set comprises 4331 images. The performance diagrams of the training and test accuracy, as well as the training and test losses, along with the confusion matrix, are shown in Fig. 10.

The confusion matrix indicated instances of misclassification errors. For instance, there is confusion between using the right and left hands to make a phone call and sending a text. This may be attributed to the similarity in hand movements when making phone calls or sending texts. Other actions may also be incorrectly identified as safe driving, possibly because of the numerous erroneous labels present in the AUC dataset, as illustrated in Fig. 11. 'Actual' represents the correct labels, while 'Label' signifies the labeled label.

Abouelnaga et al. [20] proposed an ensemble of five networks, achieving an accuracy of 95.98% on the AUC dataset. However, the network's parameter count was significantly large, at 120 million. Baheti et al. [26] introduced a modified VGG architecture that reduced the parameter count from 120 million to 15 million, achieving an accuracy of 95.54%. Currently, a few lightweight networks specifically designed for deployment on mobile devices, such as MobileNet and MobileNetV2, etc., are widely used. We compared our model with MobileNet, MobileNetV2, NasNet Mobile, SqueezeNet, MobileVGG, D-HCNN, etc., as they exhibited excellent performance in terms of both accuracy and parameter count, as shown in Table 3.

The proposed network achieved an accuracy comparable to that of AlexNet, VGGNet, MobileNet, MobileVGG, etc., with a parameter count of only 1.7 million. CCNET surpasses recent lightweight architectures (DenseNet, MobileNet, MobileNetV2, NasNet Mobile, MobileVGG, MobileNetV2-A) in accuracy with a smaller parameter count. It has a 0.45 million larger parameter count than that of SqueezeNet, but achieves a 2.31% higher accuracy. CCNET has a 1.26% lower accuracy than that of CAT-CapsNet, but comprises only 20% of the parameters of CAT-CapsNet. It achieves an accuracy comparable to that of DHCNN, but DHCNN requires HOG images alongside RGB images for input. Consequently, the computation of the HOG features for each image is required when utilizing D-HCNN, posing challenges for real-world applications. The experiments demonstrate that the model presented in this



(a) accuracy (b) loss

(c) confusion matrix

Fig. 10. Performance visualizations of CCNET on the AUC dataset: (a) training and test accuracies, (b) training and test losses, and (c) confusion matrix.



Fig. 11. Some mislabeled images from the AUC distracted driver detection dataset. First is the true class label, and second is the image label in the dataset.

paper utilizes NAS to optimize the search space and automatically select the most suitable network architecture for distracted driver behavior recognition. Additionally, the model incorporates the CA mechanism, which enhances its ability to model both global and local features, thereby improving classification accuracy.

We observed that CCNET achieved approximately 100% accuracy on the State Farm dataset, surpassing these lightweight models, whereas AUC achieved only 95.52% accuracy. This was because the State Farm dataset was accurately labeled, whereas the AUC training and validation sets contained erroneous data labels. Training on mislabeled data diminishes the model performance.

Accuracy is an intuitive metric. However, its main drawback lies in its unsuitability for imbalanced data. Considering the inherent imbalance in AUC, we present metrics such as precision, recall, F1-Score, and support for each class in Table 4. The specific classes are as follows: C0 represents Safe driving, C1 Text Right, C2 Talk Right, C3 Text Left, C4 Talk Left, C5 Adjust Radio, C6 Drink, C7 Reach Behind, C8 Hair and Makeup; and C9 Talk Passengers. Precision refers to the proportion of correctly predicted positive samples among all samples predicted as positive. Recall indicates the proportion of correctly predicted positive samples among all actual positive samples. The F1-Score is the harmonic mean of precision and recall.Precision reflects the model's ability to avoid

Table 3
Comparison of state-of-the-art models from literature on AUC dataset.

Model	Image source	Parameter size	Accuracy
AlexNet [20]	Original	62M	93.65%
	Skin Segmented	62M	93.60%
	Face	62M	86.68%
	Hands	62M	89.52%
	Face+Hands	62M	86.68%
Inception V3 [20]	Original	24M	95.17%
	Skin Segmented	24M	94.57%
	Face	24M	88.82%
	Hands	24M	91.62%
	Face+Hands	24M	90.88%
Majority Voting Ensemble [20]	–	120M	95.77%
GA-Weighted Ensemble [20]	–	120M	95.98%
Original VGG [22]	Original	140M	94.44%
VGG with Regularization [22]	Original	140M	96.31%
Modified VGG [22]	Original	15M	95.54%
DenseNet+Latent+Pose [41]	Original	8.06M	94.20%
MobileNet [23]	Original	4.2M	94.67%
MobileNetV2 [24]	Original	3.5M	94.74%
NasNet Mobile [32]	Original	5.3M	94.69%
SqueezeNet [25]	Original	1.25M	93.21%
MobileVGG [26]	Original	2.2M	95.24%
D-HCNN [27]	HOG	0.76M	95.59%
CAT-CapsNet [29]	Original	8.5M	96.78%
MobileNetV2-CA [30]	Original	2.24M	93.77%
CCNET	**Original**	**1.7M**	**95.52%**

Table 4
Performance metrics of the proposed CCNET architecture on the AUC dataset ('↑' indicates higher is better).

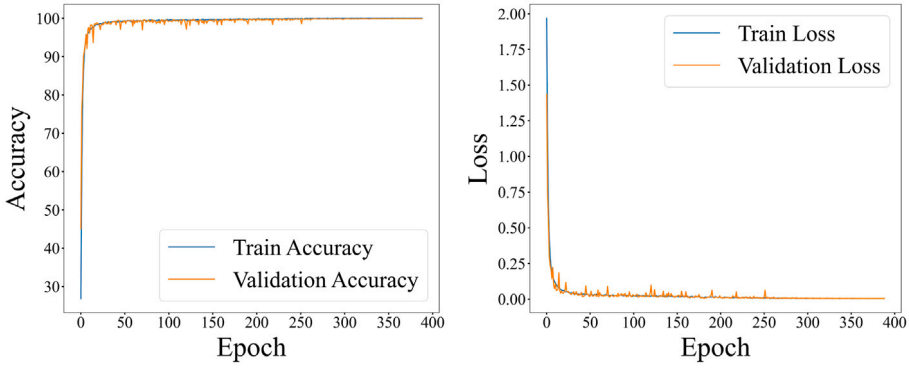
	Class	Precision(%) ↑	Recall(%) ↑	F1-Score(%) ↑	Support
	C0	91.15	96.10	93.56	922
	C1	96.31	96.01	96.16	326
	C2	98.20	96.19	97.19	341
	C3	95.03	96.76	95.89	494
	C4	97.95	93.79	95.83	306
	C5	96.68	95.41	96.04	305
	C6	95.62	97.52	96.56	403
	C7	97.52	91.36	94.34	301
	C8	99.63	92.07	95.70	290
	C9	96.42	96.27	96.34	643
Accuracy				95.52	4331

misclassifying negative samples as positive. The higher the precision, the more reliable the model is when predicting a sample as positive. Recall reflects the model's ability to identify positive samples, with higher recall indicating stronger ability to detect positive cases. The F1-score synthesizes precision and recall, offering a balanced measure of the model's performance. Support represents the total number of samples or the count of samples for each classification.

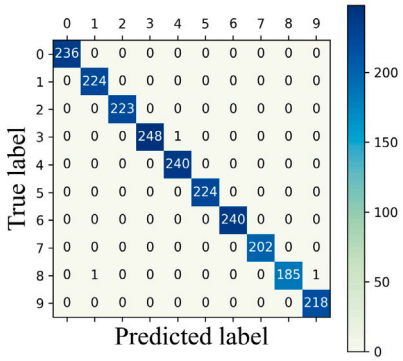
5.2.3. State Farm data partition 1

Because the validation set of State Farm lacks labels, we followed the approach of Okon and Meng [42]. The State Farm dataset was randomly partitioned into training set (90%) and testing sets (10%). The performance diagrams of the training and test accuracy, as well as the training and test losses, along with the confusion matrix, are shown in Fig. 12. In the figure, the accuracy is employed as a performance metric to represent the proportion of correct predictions by the model. Additionally, the loss function was used to measure the deviation of the model predictions from the actual values, which the model aims to minimize during the training process.

The experimental results are presented in Table 5. Our proposed model has the smallest number of parameters, excluding DHCNN, with only 1.7 million, and achieves the highest accuracy of 99.99%. This is because the model, which was designed based on the characteristics of the driver behavior recognition task, utilizes NAS to select the most promising and suitable model for this task. By incorporating the CA mechanism, the model achieved a high accuracy in driver behavior recognition. Additionally, owing to the correct labeling of the dataset, the model achieved an accuracy of 99.99%.



(a) accuracy (b) loss



(c) confusion matrix

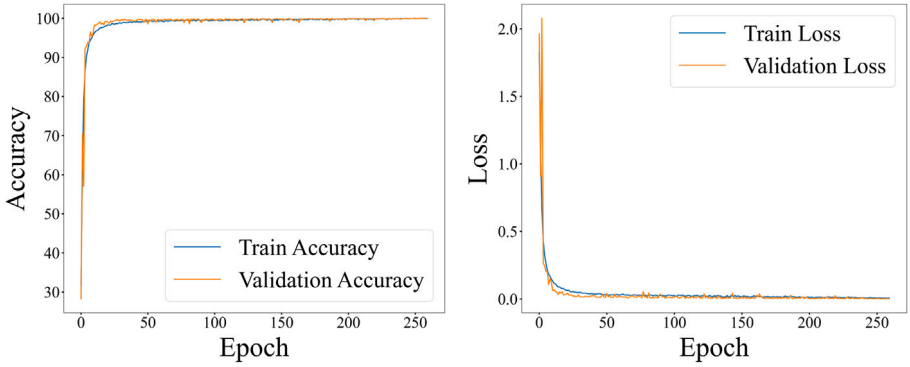
Fig. 12. Performance diagrams for CCNET on the State Farm dataset randomly split at 90%–10%: (a) training and testing accuracies, (b) training and testing losses, and (c) confusion matrix.

Table 5
Comparison of results with state-of-the-art approaches on the State Farm dataset randomly split at 90%–10%.

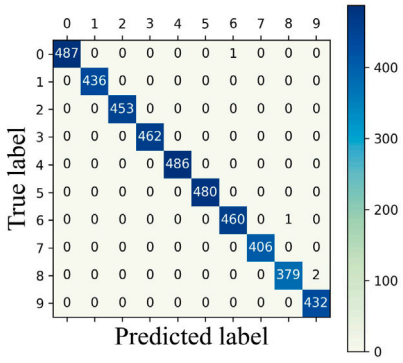
Model	Parameters size	Accuracy
AlexNet+Softmax [42]	63.2M	96.8%
AlexNet+TripleLoss [42]	63.2M	98.7%
D-HCNN [27]	0.76M	99.87%
MobileNetV2 [43]	3.5M	99.62%
CDCNN [44]	–	99.73%
MobileNetV2-tiny [43]	2.8M	99.81%
CAT-CapsNet [29]	8.5M	99.76%
MobileNetV2-CA [30]	2.24M	93.53%
CCNET	**1.7M**	**99.99%**

5.2.4. State Farm data partition 2

In this section, the State Farm dataset is randomly split into training and testing sets at 80%–20%, following the approach suggested by Zhang [45], Majdi et al. [46], and Jamsheed et al. [47]. The performance diagrams of the training and test accuracy, as well as the training and test losses and confusion matrix are shown in Fig. 13. Because the early stopping technique was employed in this study, training was terminated prematurely when the model reached convergence. As indicated in Table 6, the proposed model achieved the highest accuracy of 99.99% with a parameter count of only 1.7 million. This accuracy is 0.13% higher than that of DHCNN. It is noteworthy that CCNET uses raw images as input, whereas DHCNN requires HOG images. Hence, it is necessary to compute the HOG features for each image, which makes them unsuitable for a real-time driver behavior recognition model.



(a) accuracy (b) loss



(c) confusion matrix

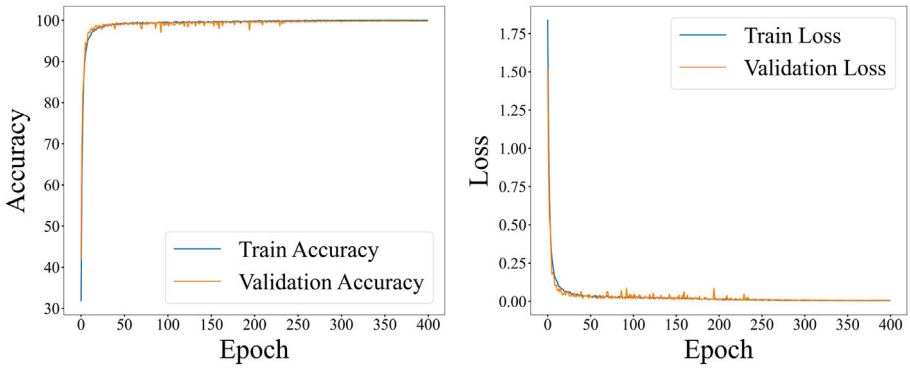
Fig. 13. Performance diagrams for CCNET on the State Farm dataset randomly split at 80%–20%: (a) training and testing accuracies, (b) training and testing losses, and (c) confusion matrix.

Table 6
Comparison of results with state-of-the-art approaches on the State Farm dataset randomly split at 80%–20%.

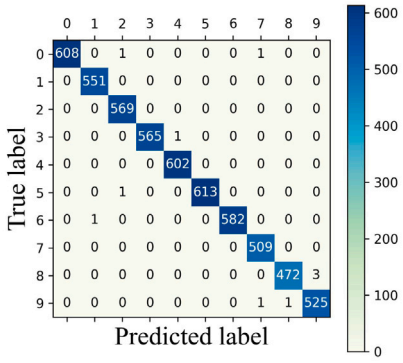
Model	Parameters size	Accuracy
Pixel SVC [45]	–	18.3%
SVC+HOG [45]	–	28.2%
SVC+PCA [45]	–	34.8%
SVC+Bbox+PCA [45]	–	40.7%
Original VGG-16 [45]	140M	90.2%
VGG-GAP [45]	140M	98.7%
Original VGG-16+ VGG-GAP [45]	>140M	92.6%
MLP [46]	–	82.0%
RNN [46]	–	91.7%
Driver-Net [46]	–	95.0%
Vanilla CNN with Data Augmentation [47]	26.05M	97.5%
CNN with Transfer Learning [47]	>3.5M	71.72%
HCF [48]	>72.3M	96.74%
D-HCNN [27]	0.76M	99.86%
CAT-CapsNet [29]	8.5M	99.88%
CCNET	**1.7M**	**99.99%**

5.2.5. State Farm data partition 3

Masood et al. [49] randomly divided the State Farm training dataset into training and testing sets at 75%–25% and adopted the approach. The performance diagrams of the training and test accuracies, training and test losses, and confusion matrix, are shown in Fig. 14.



(a) accuracy (b) loss



(c) confusion matrix

Fig. 14. Performance diagrams for CCNET on the State Farm dataset randomly split at 75%–25%: (a) training and testing accuracies, (b) training and testing losses, and (c) confusion matrix.

Table 7
Comparison of results with state-of-the-art approaches on the State Farm dataset randomly split at 75%–25%.

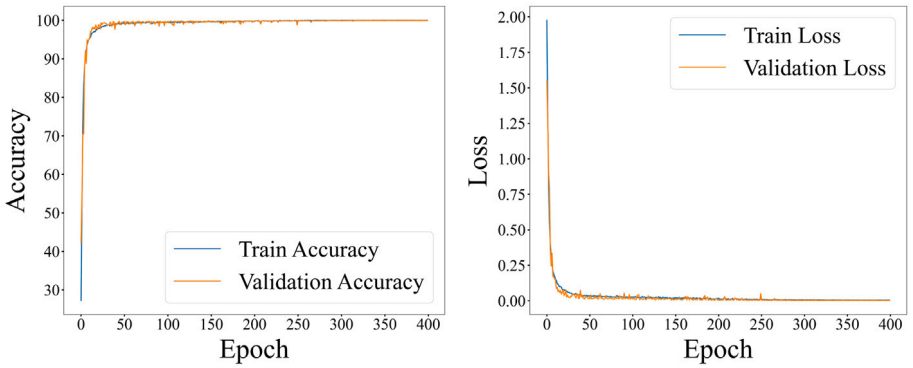
Model	Parameters size	Accuracy
UnPretrained VGG16 [49]	140M	99.43%
Pretrained VGG16 [49]	140M	99.57%
UnPretrained VGG19 [49]	142M	98.98%
Pretrained VGG19 [49]	142M	99.39%
MobileVGG [23]	2.2M	99.75%
Transfer Learning with Resnet [19]	60M	85.00%
D-HCNN [27]	0.76M	99.84%
CCNET	**1.7M**	**99.96%**

In this experiment, the accuracy of the proposed CCNET was the highest. CCNET demonstrated an accuracy improvement of 0.21 and a reduction in parameter count by 0.5 million compared to MobileVGG, which utilizes depth-wise separable convolutions. Additionally, CCNET exhibited higher accuracy (0.12) than the DHCNN model. Other experimental results are presented in Table 7.

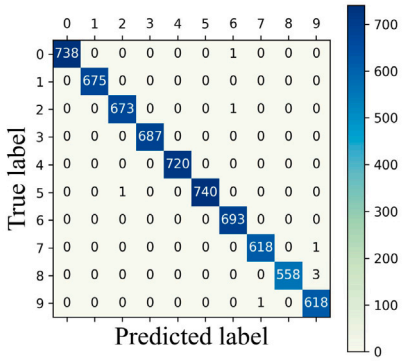
5.2.6. State Farm data partition 4

Masood et al. [50] randomly divided the State Farm training dataset into training and testing sets at 70%–30% and adopted the approach. The performance diagrams of the training and test accuracies, training and test losses, and confusion matrix, are shown in Fig. 15.

The experiments revealed that CCNET outperformed MobileNetV2 by 0.26% in terms of accuracy, with the parameters accounting for only 48.57% of the MobileNetV2. Furthermore, CCNET outperformed Inception V3 + Xception [50] by 9.93%. In addition, CCNET surpassed Inception V3 + ResNet50 + Xception + VGG-19 [50] with an accuracy of 2.93%.Other experimental results are presented in Table 8.



(a) accuracy (b) loss



(c) confusion matrix

Fig. 15. Performance diagrams for CCNET on the State Farm dataset randomly split at 70%–30%: (a) training and testing accuracies, (b) training and testing losses, and (c) confusion matrix.

Table 8
Comparison of results with state-of-the-art approaches on the State Farm dataset randomly split at 70%–30%.

Model	Parameters size	Accuracy
Original VGG-16 [50]	140M	58.30%
VGG-19 [50]	142M	55.70%
Inception V3 [50]	25.6M	92.90%
Xception [50]	22.9M	82.5%
Inception V3+Xception [50]	46.7M	90.00%
Inception V3+ResNet50+Xception+VGG-19 [50]	214.3M	97.00%
D-HCNN [27]	0.76M	99.82%
MobileNetV2 [43]	3.5M	99.57%
MobileNetV2-tiny [43]	2.78M	99.88%
CAT-CapsNet [29]	8.5M	99.50%
CCNET	**1.7M**	**99.93%**

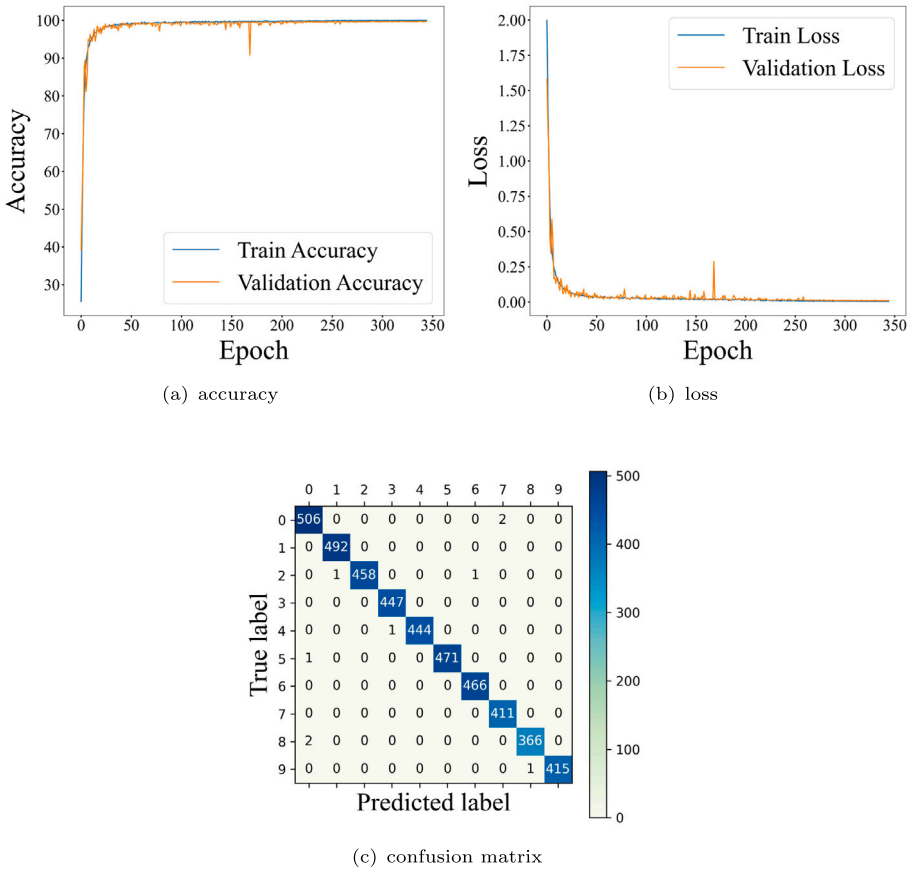
5.2.7. State Farm data partition 5

In this experiment, we followed the method of Wang and Wu [43] and split the training set of the State Farm dataset into 60% for training, 20% for validation, and 20% for testing to conduct the cross-validation experiments. The performance diagrams of the training and test accuracy, training and test losses, and confusion matrix, are shown in Fig. 16.

Our proposed CCNET exhibited an accuracy improvement of 1.1% over MobileNetV2-tiny and 3% over MobileNetV2 while having a parameter count of only 48.57% compared with MobileNetV2. The other experimental results are presented in Table 9.

5.2.8. Ablation study

In this section, the effectiveness of the CA module in the proposed model is evaluated. The experimental results, as shown in Table 10, demonstrate that removing the CA module leads to a decrease in the model performance. By contrast, adding the CA module improves the accuracy while only increasing the model's parameter count by 0.2M. It is observed that both CNET and



(a) accuracy (b) loss

(c) confusion matrix

Fig. 16. Performance diagrams for CCNET on the State Farm dataset randomly split by 60%-20%-20%: (a) training and test accuracies, (b) training and test losses, and (c) confusion matrix.

Table 9
Comparison of results with state-of-the-art approaches on the State Farm dataset randomly split at 60%–20%–20%.

Model	Parameters size	Accuracy
MobileNetV2 [43]	3.5M	96.80%
MobileNetV2-tiny [43]	2.78M	98.70%
CCNET	**1.7M**	**99.80%**

Table 10
Comparison of accuracy and parameters size with and without CA module on the AUC dataset and the State Farm dataset.

Model	Parameters size	AUC	State Farm(90%–10%)	State Farm(80%–20%)	State Farm(70%–30%)
CNET	1.72M	94.25%	99.95%	99.88%	99.78%
CCNET	**1.74M**	**95.52%**	**99.99%**	**99.99%**	**99.93%**

CCNET exhibit a decline in accuracy as the ratio of the training set to the validation set in the State Farm dataset decreases. This is because a smaller training set means fewer samples for the models to learn from, which can cause the models to inadequately learn the features of the data and thus result in lower accuracy.

5.2.9. CCNET architecture real-time test and flops

The CCNET model demonstrated superior real-time performance compared to other lightweight models. The remaining results are listed in Table 11. Parameters Size represents the total number of parameters in the model. Single Image refers to the time required to process a single image; a shorter time indicates higher inference efficiency of the model. FPS measures the number of images the model can process per second; a higher FPS indicates better performance. Floating point operations per second (FLOPS) quantifies the computational complexity required for forward or backward propagation; higher FLOPS values signify a more complex model. The table reveals that when processing a single image in test mode, the CCNET model requires 12 FLOPs and 2.88 ms on

Table 11
Comparison of CCNET with lightweight networks in terms of FLOPS and time consumption ('↑' indicates higher is better,'↓' indicates lower is better).

Model	Parameters size ↓	Single image↓	Frame Per Second (FPS)↑	FLOPS(M)↓
MobileNetV2	3.5M	5.12 ms	194	327
mobilenet_v3_small	2.54M	4.52 ms	221	62
mobilenet_v3_large	5.48M	5.70 ms	175	235
CCNET	**1.7M**	**2.88 ms**	**348**	**12**

NVIDIA GeForce RTX 3060. In comparison, MobileNetV2 requires 327 FLOPs and 5.12 ms. MobileNet_V3_Small requires 62 FLOPs and 4.52 ms. MobileNet_V3_Large requires 235 FLOPs and 5.70 ms. This is because the network in this study employs CondConv, utilizing this convolution to increase the model's capacity while maintaining efficient inference speed and lower computational requirements during inference. To simulate real driving scenarios, the model was deployed on an embedded device (Nvidia Jetson Nano B01), achieving a real processing speed of 23 FPS. To deploy it more efficiently on embedded devices, we further optimized it using the TensorRT framework, achieving 54 FPS.

6. Conclusion

In this study, we designed a neural architecture search space tailored to the characteristics of distracted driver behavior tasks, identifying the most suitable network for recognizing such behaviors, named CNET. By integrating this with CA, we created a lightweight framework specifically for distracted driving recognition, referred to as CCNET. Our model outperformed several contemporary architectures in terms of accuracy and size, achieving accuracies of 95.52% and exceeding 99.8% on the AUC and State Farm public datasets, respectively, while maintaining a parameter count of only 1.7 million. Furthermore, when deployed on an Nvidia Jetson Nano B01, CCNET achieved a real-time processing speed of 54 FPS, facilitating real-time analysis in simulated driving scenarios. However, our model does have limitations, including its reliance on specific datasets that may not encompass the full spectrum of distracted driving behaviors and its performance in varied real-world conditions. Future work will focus on expanding the neural network search space to explore additional architectures, adapting CCNET for video datasets to leverage temporal information, and investigating data fusion techniques to integrate vehicle speed and other sensor data for enhanced predictive accuracy.

CRediT authorship contribution statement

Haibin Sun: The designer of method, Writer of paper, The major contributor of this paper. **Mengting Zhang:** Provides valuable suggestions and assistance for this paper.

Ethical and informed consent for data used

Used data are open-source and have no associated privacy and copyright issues.

Funding statement

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that there is no conflict of interest regarding the publication of this paper.

Data availability

Both the AUC public dataset and the StateFarm open training dataset were used for this study. Data availability: Previously reported [StateFarm Distracted Driver Detection] data were used to support this study and are available at https://kaggle.com/competitions/state-farm-distracted-driver-detection. Previously reported [AUC Distracted Driver (AUCD2)] data were used to support this study and are available at https://heshameraqi.github.io/distraction_detection.

References

[1] Ministry of road transport and highways releases annual report on “road accidents in India-2022”. 2023, URL https://pib.gov.in/Pressreleaseshare.aspx?PRID=1973295. [Accessed 23 November 2024].
[2] Lotfipour S, Mortazavi R, Chakravarthy B. National highway traffic safety administration (NHTSA) notes. Ann Emerg Med 2007;49:535–7.
[3] Cellan-Jones R. Uber’s self-driving operator charged over fatal crash. BBC News 2020.
[4] Krizhevsky A, Sutskever I, Hinton GE. ImageNet classification with deep convolutional neural networks. Commun ACM 2017;60(6):84–90.
[5] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition. 2014, arXiv preprint arXiv:1409.1556.
[6] Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A. Going deeper with convolutions. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2015, p. 1–9.
[7] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2016, p. 770–8.
[8] Szegedy C, Ioffe S, Vanhoucke V, Alemi A. Inception-v4, inception-resnet and the impact of residual connections on learning. In: Proceedings of the AAAI conference on artificial intelligence. 31, 2017.
[9] Elsken T, Metzen JH, Hutter F. Neural architecture search: A survey. J Mach Learn Res 2019;20(1):1997–2017.
[10] Hou Q, Zhou D, Feng J. Coordinate attention for efficient mobile network design. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2021, p. 13713–22.
[11] Distracted driving dangers and statistics | NHTSA. 2023, https://www.nhtsa.gov/risky-driving/distracted-driving. [Accessed 07 December 2023].
[12] Zhang X, Zheng N, Wang F, He Y. Visual recognition of driver hand-held cell phone use based on hidden CRF. In: Proceedings of 2011 IEEE international conference on vehicular electronics and safety. IEEE; 2011, p. 248–51.
[13] Das N, Ohn-Bar E, Trivedi MM. On performance evaluation of driver hand detection algorithms: Challenges, dataset, and metrics. In: 2015 IEEE 18th international conference on intelligent transportation systems. IEEE; 2015, p. 2953–8.
[14] Seshadri K, Juefei-Xu F, Pal DK, Savvides M, Thor CP. Driver cell phone usage detection on strategic highway research program (SHRP2) face view videos. In: Proceedings of the IEEE conference on computer vision and pattern recognition workshops. 2015, p. 35–43.
[15] Hoang Ngan Le T, Zheng Y, Zhu C, Luu K, Savvides M. Multiple scale faster-rcnn approach to driver’s cell-phone usage and hands on steering wheel detection. In: Proceedings of the IEEE conference on computer vision and pattern recognition workshops. 2016, p. 46–53.
[16] Zhao C, Zhang B, He J, Lian J. Recognition of driving postures by contourlet transform and random forests. IET Intell Transp Syst 2012;6(2):161–8.
[17] Yan C, Coenen F, Zhang B. Driving posture recognition by convolutional neural networks. IET Comput Vis 2016;10(2):103–14.
[18] State farm distracted driver detection. 2016, https://kaggle.com/competitions/state-farm-distracted-driver-detection. [Accessed 08 November 2023].
[19] Hssayeni MD, Saxena S, Ptucha R, Savakis A. Distracted driver detection: Deep learning vs handcrafted features. Electron Imaging 2017;29:20–6.
[20] Abouelnaga Y, Eraqi HM, Moustafa MN. Real-time distracted driver posture classification. 2017, arXiv preprint arXiv:1706.09498.
[21] Baheti B, Gajre S, Talbar S. Detection of distracted driver using convolutional neural network. In: Proceedings of the IEEE conference on computer vision and pattern recognition workshops. 2018, p. 1032–8.
[22] Zhao Z, Xia S, Xu X, Zhang L, Yan H, Xu Y, Zhang Z. Driver distraction detection method based on continuous head pose estimation. Comput Intell Neurosci 2020;2020:1–10.
[23] Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, Andreetto M, Adam H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. 2017, arXiv preprint arXiv:1704.04861.
[24] Sandler M, Howard A, Zhu M, Zhmoginov A, Chen L-C. Mobilenetv2: Inverted residuals and linear bottlenecks. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2018, p. 4510–20.
[25] Iandola FN, Han S, Moskewicz MW, Ashraf K, Dally WJ, Keutzer K. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 MB model size. 2016, arXiv preprint arXiv:1602.07360.
[26] Baheti B, Talbar S, Gajre S. Towards computationally efficient and realtime distracted driver detection with mobilevgg network. IEEE Trans Intell Veh 2020;5(4):565–74.
[27] Qin B, Qian J, Xin Y, Liu B, Dong Y. Distracted driver detection based on a CNN with decreasing filter size. IEEE Trans Intell Transp Syst 2021;23(7):6922–33.
[28] Chen D, Wang Z, Wang J, Shi L, Zhang M, Zhou Y. Detection of distracted driving via edge artificial intelligence. Comput Electr Eng 2023;111:108951.
[29] Mittal H, Verma B. CAT-CapsNet: A convolutional and attention based capsule network to detect the driver’s distraction. IEEE Trans Intell Transp Syst 2023;24(9):9561–70.
[30] Gao H, Liu Y. Improving real-time driver distraction detection via constrained attention mechanism. Eng Appl Artif Intell 2024;128:107408.
[31] Yang B, Bender G, Le QV, Ngiam J. Condconv: Conditionally parameterized convolutions for efficient inference. Adv Neural Inf Process Syst 2019;32.
[32] Zoph B, Vasudevan V, Shlens J, Le QV. Learning transferable architectures for scalable image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2018, p. 8697–710.
[33] Liu C, Zoph B, Neumann M, Shlens J, Hua W, Li L-J, Fei-Fei L, Yuille A, Huang J, Murphy K. Progressive neural architecture search. In: Proceedings of the European conference on computer vision. ECCV, 2018, p. 19–34.
[34] Real E, Aggarwal A, Huang Y, Le QV. Regularized evolution for image classifier architecture search. In: Proceedings of the aaai conference on artificial intelligence. Vol. 33, 2019, p. 4780–9.
[35] Liu H, Simonyan K, Yang Y. Darts: Differentiable architecture search. 2018, arXiv preprint arXiv:1806.09055.
[36] Wang X. Teacher guided neural architecture search for face recognition. In: Proceedings of the AAAI conference on artificial intelligence. Vol. 35, 2021, p. 2817–25.
[37] Hu J, Shen L, Sun G. Squeeze-and-excitation networks. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2018, p. 7132–41.
[38] Woo S, Park J, Lee J-Y, Kweon IS. Cbam: Convolutional block attention module. In: Proceedings of the European conference on computer vision. ECCV, 2018, p. 3–19.
[39] Chen J, Jiang Y, Huang Z, Guo X, Wu B, Sun L, Wu T. Fine-grained detection of driver distraction based on neural architecture search. IEEE Trans Intell Transp Syst 2021;22(9):5783–801.
[40] Loshchilov I, Hutter F. Stochastic gradient descent with warm restarts. In: Proceedings of the 5th int. conf. learning representations. p. 1–16.
[41] Behera A, Keidel AH. Latent body-pose guided DenseNet for recognizing driver’s fine-grained secondary activities. In: 2018 15th IEEE international conference on advanced video and signal based surveillance. AVSS, IEEE; 2018, p. 1–6.
[42] Okon OD, Meng L. Detecting distracted driving with deep learning. In: Interactive collaborative robotics: second international conference, ICR 2017, hatfield, UK, September 12-16, 2017, proceedings 2. Springer; 2017, p. 170–9.
[43] Wang J, Wu Z, et al. Model lightweighting for real-time distraction detection on resource-limited devices. Comput Intell Neurosci 2022;2022.
[44] Gumaei A, Al-Rakhami M, Hassan MM, Alamri A, Alhussein M, Razzaque MA, Fortino G. A deep learning-based driver distraction identification framework over edge cloud. Neural Comput Appl 2020;1–16.

[45] Zhang B. Apply and compare different classical image classification method: Detect distracted driver. Project rep 229, Stanford, CA, USA: Dept. Comput. Sci., Stanford Univ.; 2016.
[46] Majdi MS, Ram S, Gill JT, Rodríguez JJ. Drive-net: Convolutional network for driver distraction detection. In: 2018 IEEE southwest symposium on image analysis and interpretation. SSIAI, IEEE; 2018, p. 1–4.
[47] Janet B, Reddy US, et al. Real time detection of driver distraction using CNN. In: 2020 third international conference on smart systems and inventive technology. ICSSIT, IEEE; 2020, p. 185–91.
[48] Huang C, Wang X, Cao J, Wang S, Zhang Y. HCF: A hybrid CNN framework for behavior detection of distracted drivers. IEEE Access 2020;8:109335–49.
[49] Masood S, Rai A, Aggarwal A, Doja MN, Ahmad M. Detecting distraction of drivers using convolutional neural network. Pattern Recognit Lett 2020;139:79–85.
[50] Dhakate KR, Dash R. Distracted driver detection using stacking ensemble. In: 2020 IEEE international students' conference on electrical, electronics and computer science. SCEECS, IEEE; 2020, p. 1–5.